



UNIVERSIDAD INTERNACIONAL DEL ECUADOR
ING. CIBERSEGURIDAD - EN LINEA

ASIGNATURA:

Lógica de programación

TEMA:

Aprendizaje autónomo 1

CICLO:

PRIMERO

ESTUDIANTE:

ANGEL GABREL YUMBLA OCHOA

DOCENTE:

MONICA PATRICIA SALAZAR TAPIA

FECHA DE ENTREGA:

31 de mayo del 2026

INTRODUCCION.

El siguiente trabajo tiene como objetivo elaborar un diseño previo de un programa informático mediante diagramas y arquitectura de software. Para dicha actividad se presento el juego “adivina el numero”, donde el computador intenta adivinar un numero pensando por el usuario. Antes de empezar el desarrollo del código, es importante comprender el problema ha tratar, identificar las principales funciones del sistema y dar representación a su estructura como tal. Para ello, se producirá un diagrama de caso de uso para visualizar la interacción entre el usuario y el sistema, y un diagrama de arquitectura para representar los componentes principales del software.

DESARROLLO.

Diagramas de funcionalidad (caso de uso)

Diagrama de casos de uso

Un diagrama de casos de uso es como un plano de la funcionalidad de tu sistema. Representa visualmente las interacciones entre los usuarios (o "actores") y el sistema, destacando las diversas formas en que se puede utilizar. Imagínalo como un mapa general que muestra las diferentes rutas que los usuarios pueden seguir para alcanzar sus objetivos dentro del sistema. Este mapa ayuda a todos los involucrados, desde desarrolladores hasta partes interesadas, a comprender claramente las capacidades e interacciones del sistema. Al proporcionar una visión general visual, los diagramas de casos de uso hacen que los sistemas complejos sean más accesibles y fáciles de entender. Son una herramienta crucial en las primeras etapas del diseño y desarrollo de sistemas, ya que garantizan que todas las partes interesadas tengan una comprensión unificada de cómo funcionará el sistema.

Componentes de un diagrama de casos de uso

Desglosar un diagrama de casos de uso en sus componentes fundamentales puede simplificar su estructura y facilitar su creación y comprensión. Esto es lo que necesitas saber:

Actores

Los actores son las entidades que interactúan con el sistema. Pueden ser usuarios, otros sistemas o incluso hardware. Por ejemplo, en una aplicación bancaria, los actores podrían incluir clientes, cajeros y los sistemas internos del banco.

Actores principales: Son los usuarios o entidades que interactúan directamente con el sistema para lograr un objetivo.

Actores secundarios: Se trata de las entidades que prestan servicios al sistema, a menudo involucradas indirectamente en el caso de uso principal.

Sistemas externos: Se trata de sistemas ajenos a la propia plataforma que interactúan con ella, como por ejemplo las pasarelas de pago o las API de terceros.

Casos de uso

Los casos de uso son las acciones o servicios específicos que el sistema proporciona a los actores. Cada caso de uso representa un objetivo que un actor puede alcanzar al interactuar con el sistema.

- **Funciones principales:** Son las acciones básicas que admite el sistema, como "Retirar dinero" o "Consultar saldo" en un sistema bancario.
- **Subfunciones:** Son funciones secundarias que dan soporte a las acciones principales, como "Imprimir recibo" después de retirar dinero.
- **Manejo de excepciones:** Estos casos de uso manejan las excepciones o errores que puedan ocurrir durante un caso de uso principal, como por ejemplo "Fondos insuficientes".

Asociaciones

Las asociaciones son las líneas que conectan a los actores con los casos de uso. Ilustran sus interacciones. Por ejemplo, una línea entre un actor cliente y el caso de uso "Retirar dinero" muestra que los clientes pueden retirar dinero del sistema bancario.

- **Asociaciones directas:** Interacciones sencillas entre actores y casos de uso.
- **Asociaciones indirectas:** Implican actores o sistemas secundarios que apoyan la interacción.

Límite del sistema

El límite del sistema define qué elementos se incluyen en el diagrama. Se representa mediante un rectángulo que engloba todos los casos de uso. Todo lo que se encuentre fuera de este límite no forma parte del sistema, pero puede interactuar con él.

- **Alcance del sistema:** Define los límites del sistema.
- **Interacciones externas:** Identifica las interacciones con agentes externos al sistema.

Vías de comunicación

Las rutas de comunicación muestran cómo interactúan los actores y los casos de uso. Ayudan a clarificar el flujo de información o acciones entre los actores y el sistema. Estas rutas son esenciales para comprender y optimizar las interacciones del sistema.

- **Flujo de datos:** Muestra cómo se transmiten los datos entre los actores y los casos de uso.
- **Diagrama de flujo de acciones:** Ilustra la secuencia de acciones u operaciones que ocurren. (Equipo Miro, 2025)

Diagrama de arquitectura de software

La diagramación arquitectónica de software consiste en crear mapas visuales de las partes de tu sistema de software. Estos diagramas muestran qué hace cada pieza, cómo está construida y cómo encaja todo.

Piensa en los diagramas de arquitectura de software como planos de tu software. Facilitan la toma de decisiones y la comprensión incluso de las configuraciones de software más complejas. Al exponerlo todo visualmente, puedes detectar posibles problemas con antelación, agilizar la comunicación entre tu equipo y mantener a todo el mundo en la misma línea. (El Equipo Miro, 2025)

Los diagramas de arquitectura ofrecen varias ventajas, como la posibilidad de colaborar, reducir los riesgos, aumentar la eficacia y la escalabilidad.

Colaboración

Los diagramas de arquitecturas favorecen considerablemente la colaboración entre desarrolladores y diseñadores, a la vez que proporcionan una visión unificada de la funcionalidad del sistema y de los posibles problemas. Un entendimiento compartido de un sistema, aplicación o sitio web aporta varias ventajas. Facilita la comunicación durante el proceso de diseño, ayuda a los equipos a desarrollar componentes de software de sistemas eficaces y garantiza el cumplimiento de los objetivos de un proyecto.

Reducción del riesgo

Los diagramas de arquitectura identifican riesgos potenciales en el desarrollo del sistema, como suposiciones incorrectas, lógica defectuosa o pruebas inadecuadas. Al identificar y abordar los riesgos en una fase temprana del ciclo de vida de desarrollo del software, los equipos de desarrollo son capaces de realizar cambios antes y así reducir el riesgo de que aparezcan problemas importantes en una fase posterior.

Eficiencia

Los diagramas de arquitectura proporcionan una visión clara de los componentes y la estructura del sistema. De este modo, las partes interesadas pueden identificar los problemas con precisión y resolverlos rápidamente. Los diagramas también permiten mantener y escalar los sistemas con mayor facilidad, de modo que los cambios continuos resultan más eficaces.

Escalabilidad

Los diagramas de arquitectura permiten a las partes interesadas identificar formas eficientes de escalar un sistema. Por ejemplo, un diagrama puede mostrar si la arquitectura de un sistema es

centralizada o distribuida. Dado que los componentes distribuidos escalan de forma más eficiente, los componentes monolíticos se pueden actualizar o sustituir a tiempo. De igual modo, las representaciones gráficas permiten comprender cómo se almacenan y se mueven los datos. Las partes interesadas pueden identificar posibles cuellos de botella y formas de evitarlos.

Patrones de arquitectura de software que se pueden representar por medio de diagramas de arquitecturas

Los patrones de arquitectura de software son principios de diseño y prácticas recomendadas que se utilizan para desarrollar sistemas de software. Proporcionan un marco para estructurar el software y abordar retos específicos en arquitecturas de software complejas. Estos son algunos de los patrones de arquitectura de software más utilizados.

- **Arquitectura cliente-servidor**

La arquitectura cliente-servidor es una estructura de aplicación distribuida que separa las tareas y las cargas de trabajo entre servidores y clientes. Los servidores proporcionan el recurso o servicio, mientras que los clientes lo solicitan.

Tanto el cliente como el servidor son programas independientes que se comunican a través de una red. Un navegador web y un servidor web son un ejemplo de arquitectura cliente-servidor. Se trata de una arquitectura comúnmente utilizada en la computación distribuida.

- **Arquitectura orientada a servicios**

La arquitectura orientada a servicios permite la interacción entre componentes de aplicaciones distribuidas a través de servicios. Los servicios son abstractos, débilmente acoplados y autónomos en cuanto al lenguaje. Las aplicaciones acceden a estos a través de interfaces. Los desarrolladores pueden volver a utilizar los servicios existentes en lugar de tener que volver a crearlos desde cero. Las arquitecturas orientadas a servicios se utilizan habitualmente en sistemas distribuidos, ya que los servicios se pueden implementar en varios servidores.

- **Arquitectura de microservicios**

Las arquitecturas orientadas a servicios han evolucionado aún más, por lo que los desarrolladores utilizan arquitecturas de microservicios para crear, implementar y administrar servicios individuales. Las aplicaciones se dividen en servicios que se pueden implementar de forma independiente y que se comunican a través de API.

Los servicios más pequeños e independientes permiten a los desarrolladores desarrollar, probar e implementar aplicaciones de forma más sencilla y ofrecen una mayor tolerancia a los errores y la

capacidad de escalar rápidamente. Un ejemplo de una arquitectura de microservicios es una aplicación web formada por varios servicios independientes, cada uno responsable de tareas específicas.

- **Arquitecturas centradas en la nube**

Las arquitecturas centradas en la nube se utilizan para diseñar y crear aplicaciones para entornos en la nube. Las arquitecturas centradas en la nube se crean y suministran con tecnologías específicas para la nube, como contenedores, microservicios, DevOps y computación sin servidor. Priorizan la implementación y la administración automatizadas de modo que sea posible escalar y des escalar las aplicaciones verticalmente según sea necesario.

- **Arquitectura basada en eventos**

Las arquitecturas basadas en eventos son arquitecturas de software fundamentadas en la producción, detección y consumo de eventos. Las interacciones de los usuarios, las tareas en segundo plano y otros orígenes desencadenan eventos que, a su vez, activan otras funciones. Las arquitecturas basadas en eventos permiten que las aplicaciones respondan mejor a los cambios en el sistema de software y su entorno.

- **Arquitecturas de capas**

Las arquitecturas de capas consisten en un patrón de arquitectura de software que separa las aplicaciones en capas basadas en la lógica. Este tipo de arquitectura se diseñó para simplificar aplicaciones y sistemas complejos, ya que permite dividir las tareas entre capas.

Las capas se organizan descendentemente:

- Una capa de presentación (por ejemplo, una interfaz de usuario) está en la parte superior
- Una capa empresarial se sitúa en el centro
- Una capa de datos se encuentra en la parte inferior

Las capas también se pueden estructurar jerárquicamente, lo que facilita las tareas de mantenimiento y la escalabilidad.

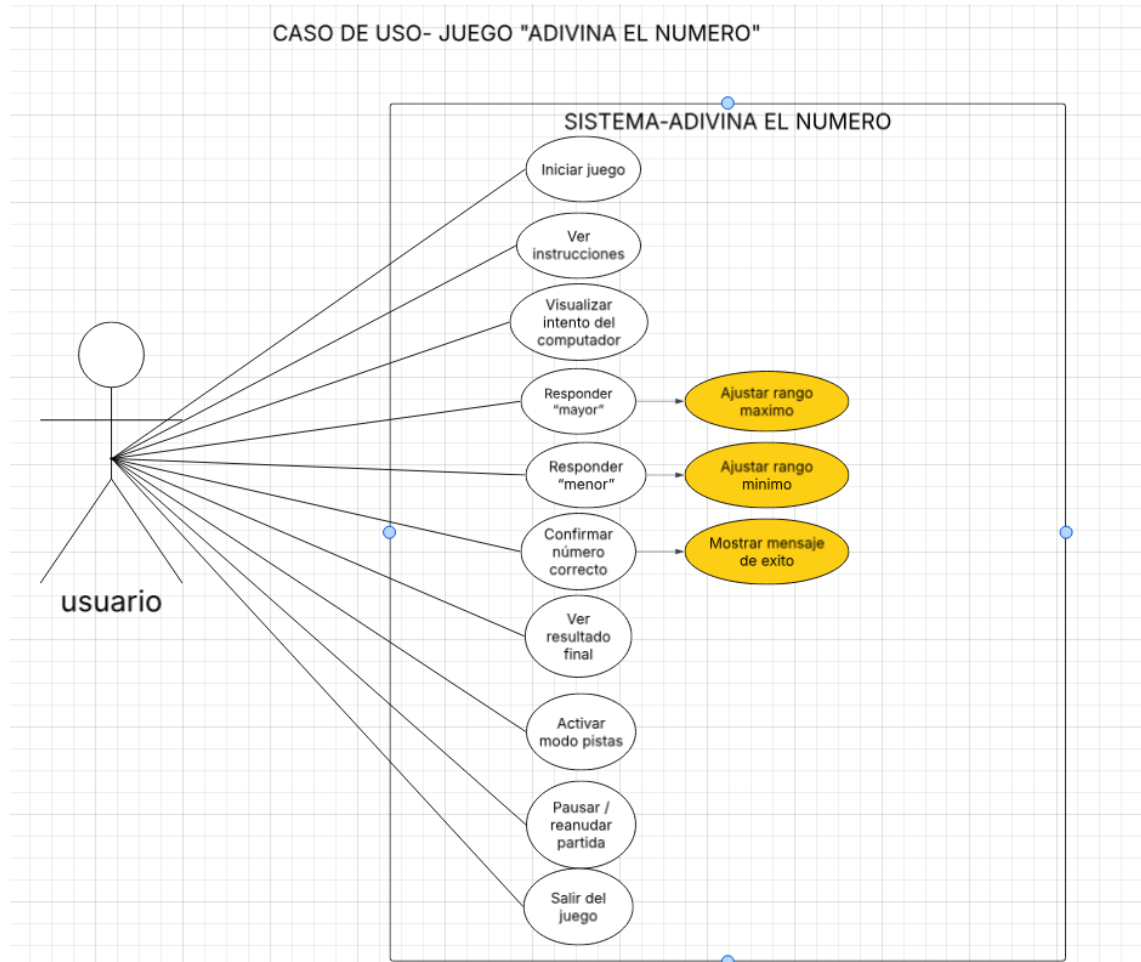
Tipo de información que se incluye en un diagrama de arquitectura

A continuación, se indica qué tipo de información es habitual encontrar en un diagrama de arquitectura:

- Los cuadrados y los círculos representan componentes, como bases de datos, redes, aplicaciones y servicios
- Las líneas y flechas muestran las conexiones e interacciones entre los componentes del sistema
- Las etiquetas proporcionan información adicional sobre los componentes y las conexiones

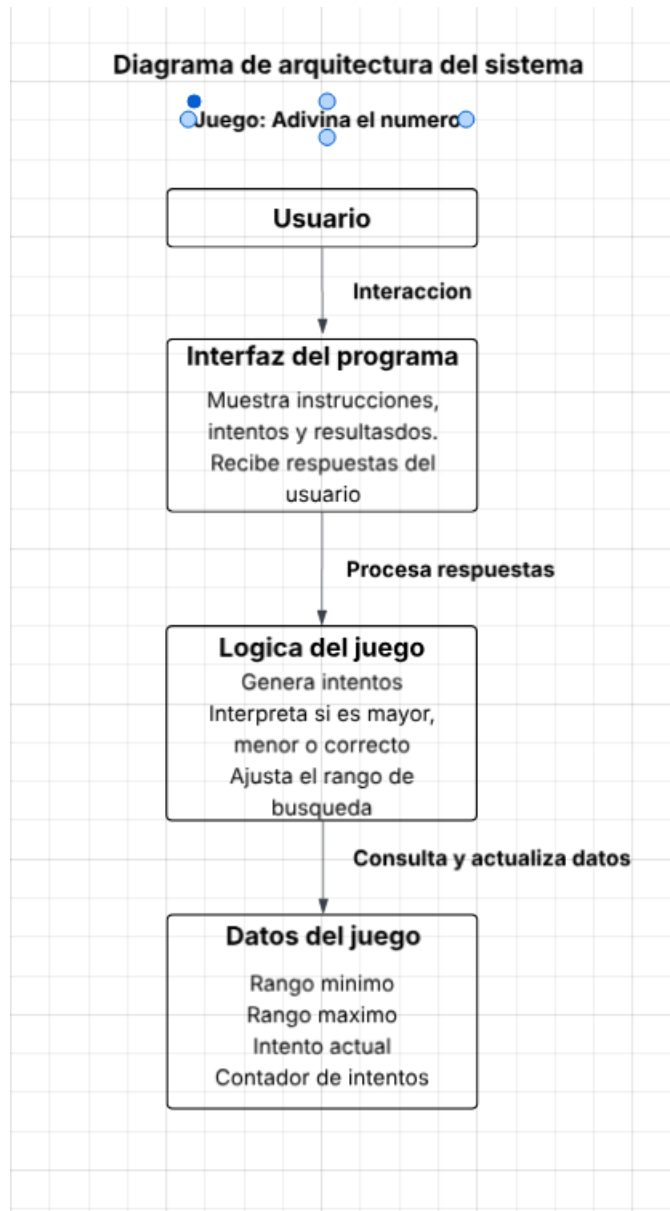
Además, es posible que el diagrama también utilice iconos o símbolos para representar visualmente los distintos componentes. Una pequeña leyenda en la parte inferior, similar a la leyenda de un mapa, explica el uso de los iconos. La forma en que se organizan los componentes y las conexiones se denomina diseño. (Amazon Web Services , 2026)

Diagrama de caso de uso



El diagrama mostrado nos muestra la interacción entre el usuario y el sistema. En este caso el usuario inicia la interacción, revisa las instrucciones y observa el número que propone el computador para luego responder si el número pensado es mayor, menor o correcto. A partir de estas respuestas el sistema continúa realizando intentos hasta encontrar el número correcto. Como conclusión el usuario visualizará el resultado de éxito al final del juego.

Diagrama de arquitectura.



La arquitectura del sistema está formada por otros componentes principales: la interfaz del programa, la lógica del juego y los datos del juego. La interfaz permite mostrar instrucciones, intentos y resultados al usuario. La lógica del juego se encarga de generar los intentos del sistema, interpretar las respuestas del usuario y ajustar el rango de búsqueda. Finalmente, los datos del juego almacenan el rango mínimo, el rango máximo, el intento actual y el contador de intentos.

conclusión.

Para concluir, el diseño del programa “Adivina el numero” facilita la comprensión del funcionamiento del sistema antes de desarrollarlo en

código. El diagrama de caso de uso nos permitió identificar las principales interacciones entre el usuario y el software, mientras que el diagrama de arquitectura nos muestra como tal toda la estructura que va a llevar dicho programa o software. Estos diagramas facilitan la planificación y la estructuración del programa, ayudan a organizar los componentes que se van a utilizar y reducen posibles fallas técnicas durante la etapa de programación.

Bibliografía

Amazon Web Services . (18 de 02 de 2026). *AWS*. Obtenido de AWS:

<https://aws.amazon.com/what-is/architecture-diagramming/>

El Equipo Miro. (10 de 10 de 2025). *miro*. Obtenido de miro:

<https://miro.com/es/diagrama/que-es-diagrama-arquitectura-software/>

Equipo Miro. (14 de 10 de 2025). *Miro*. Obtenido de Miro:

<https://miro.com/diagramming/what-is-a-use-case-diagram/>